CASE STUDY · DATA & AI

ITSM Insights

Structuring an IT support ticket flow: classification, similar-case retrieval and weak-signal detection

Client	Thales — IT Support
Programme	Ticketing (Weak signal analysis)
Prepared by	Diametral
Date	22 September 2026
Status	Design & benchmark protocol — pre-production

SCOPE OF THIS DOCUMENT

This case study consolidates the problem framing and literature review held in Notion with the technical design and experimental protocol implemented in the `itsm-insights` codebase. It describes *what was built, why each choice was made, and what has actually been measured* — separating, explicitly and throughout, results already obtained from protocols defined but not yet executed.

Contents

1. **Problem statement**
   - 1.1 Context and friction
   - 1.2 Stakeholders and pain points
   - 1.3 Objectives
2. **Literature review**
   - 2.1 Automatic classification of IT tickets
   - 2.2 Weak signals: theory and practice
   - 2.3 What the literature settles — and what it does not
3. **Design architecture**
   - 3.1 Solution overview
   - 3.2 The dummy dataset
   - 3.3 Component benchmarks
   - 3.4 Limits
4. **Conclusion**

> **READING NOTE ON EVIDENCE**
>
> Two kinds of number appear in this document and they are never mixed. **Measured** results come from evaluation pipelines that have been run on the reference dataset and are reported with their protocol. **Protocol** means the experiment is designed, implemented and runnable, but has not yet been executed — no figure is quoted for it. Section 3.3 labels every component accordingly.

1. Problem statement

The IT support team receives a steady flow of tickets that is not structured by request family. The consequence is not a reporting gap — it is an operational one: the team cannot see recurring themes, cannot prioritise on anything but arrival order, and cannot anticipate load.

1.1 Context and friction

Tickets arrive through an ITSM channel (tool, mail or form — the exact channel is still to be confirmed with the client) and follow a four-step chain:

1. **Emission** — the user files a ticket, in free text, through a channel that is not yet characterised.
2. **Reception and qualification** — the support team assigns a priority and, where one exists, a category.
3. **Handling or escalation** — resolution by the support team, or escalation to a specialised IT team.
4. **Closure** — with little or no structured capitalisation on the family or the root cause.

The friction is concentrated at steps 2 and 4. At step 2 the qualification is manual and inconsistent; at step 4 the knowledge produced by resolving the ticket is not captured in any form that a future ticket can reuse. Everything downstream — trend analysis, prioritisation, load anticipation — is blocked by those two.

This chain is a working hypothesis to be validated with the team. The open points (tool, available fields, existing categorisation) are tracked on the project's Questions page and are revisited in section 3.4.

1.2 Stakeholders and pain points

The problem is worth stating per audience, because the four stakeholders want different things and a single deliverable will not serve all of them equally.

FOR WHOM	PROBLEM	WHAT WOULD RESOLVE IT
IT support team *ticket handling*	Ticket volume not categorised in a usable way; no consolidated view of recurring families; prioritisation done as tickets arrive.	Classification into families, plus regular reporting of trends per family.
Support management *N+1 / N+2*	No forward visibility on load peaks; recurring or strained topics are discovered reactively, after the fact.	Weak signals and alerts on critical families, to anticipate incoming load.
Receiving IT teams *infra, application, network*	Repeated solicitations on similar subjects, with no capitalisation on root causes.	Identification of recurring families to direct targeted corrective or preventive action.
Internal users *ticket emitters*	Resolution time potentially lengthened when an unanticipated load peak hits.	Better anticipation of the resources to mobilise ahead of a foreseeable peak.

> **WHY THIS FRAMING MATTERS**
>
> The four rows are not one product. Agent assist (classification and suggestion), management analytics (families and alerts) and preventive action (root causes) have different buyers, different success metrics and different failure modes. Section 3.4 returns to this: attempting all three at once is a programme, whereas each one alone is a deliverable.

1.3 Objectives

The mission is phased. Phase 2 depends on Phase 1 in a hard sense: weak-signal detection on a family that has not been stabilised measures noise.

PHASE	OBJECTIVE
Phase 1 *Exploratory analysis*	Identify the format and source of the ticket history (export, ITSM database, other).
	Carry out an exploratory analysis: volumetry, seasonality, channels, priorities, receiving teams.
	Define and validate the main ticket families with the teams (classification / clustering).
	Identify, among those families, the most critical ones by business impact, recurrence and handling cost.
Phase 2 *Weak signals*	Design a weak-signal detection method per critical family (frequency evolution, trend break).
	Build an alerting model on temporal trends to anticipate load peaks.
Transverse	Put in place regular, legible reporting for management.

2. Literature review

Two bodies of work inform this project, and they are at very different stages of maturity. Ticket classification is a solved engineering problem with a well-known failure mode. Weak-signal detection on ticket volume is a much thinner literature, borrowed largely from strategic management and from AIOps vendor practice.

2.1 Automatic classification of IT tickets

MICROSOFT CSE & ENDAVA — CLASSICAL NLP PIPELINE

Problem. Helpdesk operators spent significant time manually qualifying each ticket across five properties: type, service, category, impact, urgency.

Approach. A classical NLP pipeline — text cleaning, stop-word removal, TF-IDF vectorisation — then Naive Bayes and SVM classifiers from scikit-learn, hyper-parameters tuned with grid search, served as a Flask API on Azure.

Result. About **96% accuracy** on *ticket type*, where classes are well balanced. Markedly degraded performance on *urgency* and *impact*, which are heavily imbalanced — a single dominant class representing 70 to 90% of cases.

> **KEY LESSON**
> Data quality and class balance weigh more on the outcome than the choice of algorithm. An imbalanced target makes a global metric actively misleading.

ENTERPRISE STUDY — 87 859 MULTILINGUAL TICKETS (*SYSTEMS*, 2026)

Context. A real company's ITSM history from an on-premise SQL database: tickets mostly in Lithuanian with some English, spread over 9 request types and 11 receiving teams.

Approach. A logistic regression over TF-IDF with a 10 000-term vocabulary, compared against a fine-tuned multilingual BERT. Class imbalance handled by oversampling minority classes during diagnosis, while keeping the natural distribution at test time.

Result. Logistic regression reached **macro-F1 0.78** and weighted F1 0.81, performing well on frequent categories. BERT reached a higher macro-F1, with precision and recall around **0.92** on semantically complex requests.

Authors' recommendations. Keep a final human review on ambiguous cases; prefer macro-F1 over global accuracy so that rare families are properly evaluated; reserve transformer models for semantically complex cases and keep simple models where compute is constrained.

DEVREV — THE COST OF MIS-ROUTING

This industry panorama is useful less for its method than for its quantification of the problem. An organisation handling **2 000 tickets per month with a 35% mis-routing rate** can lose the equivalent of **$329 000 per year**; each reassignment adds on average **~47 minutes** to resolution time. It distinguishes three generations of triage: keyword rules (rigid), ML/NLP classification (more accurate, but limited to classifying), then knowledge graphs and autonomous agents (contextual, able to act rather than only label).

Reported client outcomes include Bolt (mean resolution time 129.8h → 62.7h over 11 months) and Descope (−54% resolution time with no added headcount).

2.2 Weak signals: theory and practice

The notion of a *weak signal* comes from strategic management (Igor Ansoff, 1975): early information, still ambiguous or incomplete, that announces a change before it becomes a *strong* signal — that is, obvious, but usually too late to act on preventively.

Transposed to IT tickets: a discreet evolution in the frequency or nature of tickets on a given family can announce a service degradation or an incoming load peak, before it becomes a major incident or a visible team overload.

ZOHO DESK (ZIA) — ANOMALY PREDICTION ON TICKET VOLUME

The day's inbound and outbound ticket volume is compared against a trend computed over the previous 30 days. An anomaly fires only when **two criteria hold simultaneously**: a volume threshold (e.g. 25, 50, 100 tickets) *and* a relative-deviation threshold against the trend (e.g. 10%, 25%, 50%).

> **KEY LESSON**
> A weak signal is a significant deviation from a *recent baseline*, not an absolute threshold. Combining a volume floor with a relative-deviation test is what keeps the false-positive rate tolerable.

PAGERDUTY — AIOPS EVENT CORRELATION

Events are correlated by context, timing or topology to reduce noise, then fine-grained behavioural anomaly detection allows preventive intervention before user impact. The vendor claims up to **−91% alert noise**. The transferable idea is the ordering: *reduce noise first, detect second*.

IBM CLOUD PAK FOR AIOPS

Large-scale aggregation and correlation of IT events so that only relevant signals surface before root-cause analysis. Originally aimed at infrastructure telemetry rather than support tickets, but the principle — filtering a weak signal out of a voluminous, noisy stream — transposes directly.

SUPPORTBENCH — FORECASTING WITHOUT A DATA TEAM

A 13-week moving average of volume per channel, adjusted by scope growth (active accounts and users) and by pre-identified business events (product release, migration, renewal) whose impact is measured: `forecast = 13-week mean × (1 + growth) + event effect`. The forecast is then converted into staffing need.

> **KEY LESSON**
> A cheap, explainable method — a spreadsheet and roughly 30 minutes a month — already gives useful anticipation. It is the baseline any sophisticated model has to beat before it earns its complexity.

2.3 What the literature settles — and what it does not

Five conclusions were carried directly into the design:

1. **Start simple.** TF-IDF plus a linear model already performs well on frequent, well-defined families. Heavier models (BERT, LLM) are for the ambiguous or semantically complex residue — and must be measured

against the cheap baseline, not assumed to beat it.

2. **Expect imbalance and choose the metric accordingly.** Imbalance is the norm in this data, not the exception; **macro-F1** is the headline, never global accuracy.
3. **Keep a human checkpoint on low-confidence cases** rather than targeting 100% automation. This became the abstention threshold and the coverage curve of section 3.3.
4. **Detect deviation from a recent baseline**, combining two criteria, rather than crossing an absolute threshold.
5. **Quantify the current cost of mis-routing** to objectify the expected gain — the DevRev figures give an order of magnitude and a method.

> **WHAT THE LITERATURE DOES NOT SETTLE**
>
> None of the reviewed sources publishes a *false-alarm rate per series-day* for ticket-volume anomaly detection, nor a detection delay on slow ramps. Vendor claims (−91% noise, X% autonomous resolution) are not measurements one can reproduce or compare. This is precisely the gap that the benchmark protocol in section 3.3 is designed to fill for this specific corpus, with ground truth.

3. Design architecture

3.1 Solution overview

The solution is a single pipeline feeding three deliverables: classify the incoming ticket, retrieve the cases already solved the same way, and monitor each family's volume for early signs of degradation.

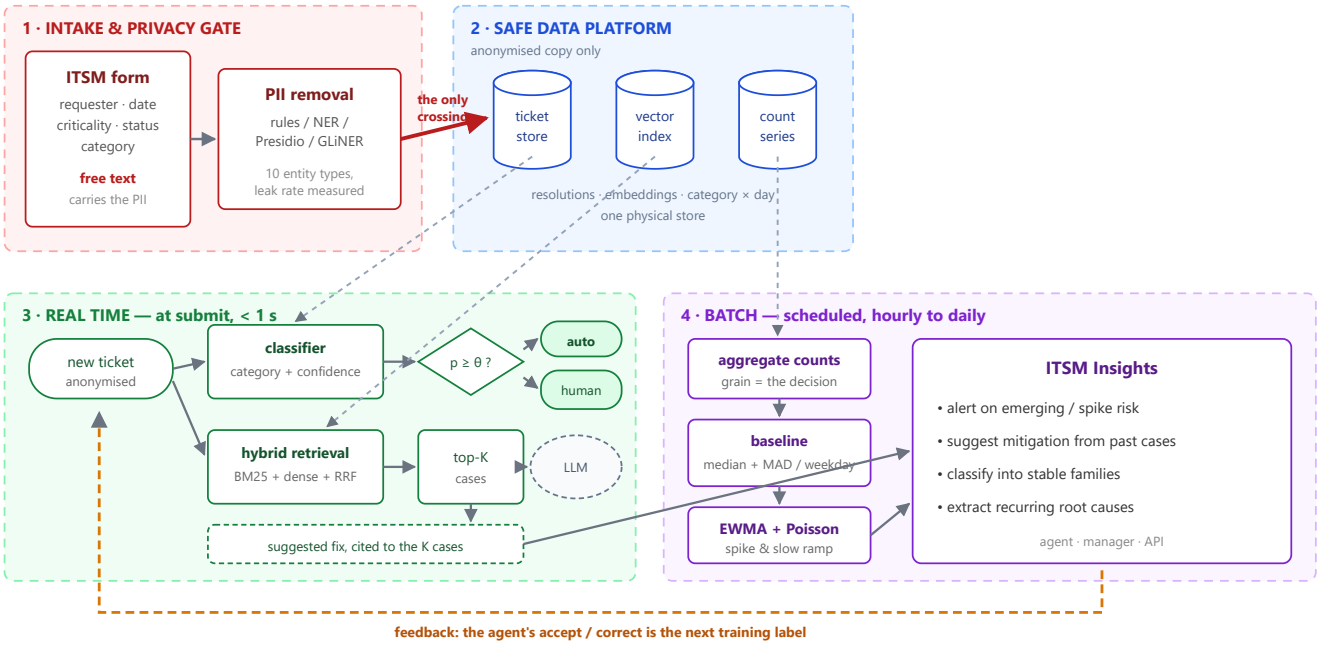


Figure 1 — Target architecture. Solid borders are implemented; dashed borders are designed and benchmarked but not yet in the shipped service.

FOUR DESIGN DECISIONS WORTH DEFENDING

The privacy gate is step zero, not a feature. Ticket free text is the one field nobody controls, and it routinely contains a colleague's name, a callback number, a mailbox, a staff number. None of it was collected on purpose, all of it is personal data, and it sits in the exact column every model reads. Anonymisation therefore sits *on* the path: it is the only door between the raw zone and everything else, and its leak rate is measured against ground truth rather than asserted.

Two clocks, not one. Suggestion is synchronous at ticket submit (under a second, availability-critical). Detection is a scheduled job (minutes, restartable). These are different runtimes with different SLAs and different failure modes; drawing them on one plane is the most common source of late surprise in this kind of design.

Abstention is part of the product. Routing is automated only above a confidence threshold; below it the ticket goes to human triage with the top-3 categories proposed. This follows directly from the literature's recommendation to keep a human checkpoint, and it is what the coverage curve in section 3.3 exists to calibrate.

The feedback loop is not optional. The agent's accept-or-correct decision returns to the store as the next training label. Without it the model never improves, production accuracy is never measured (only offline

accuracy on a frozen split), and retrieval quality remains an opinion. A reassignment after an auto-route is a free negative label and is captured too.

3.2 The dummy dataset

No client extract was available at design time, and no public corpus is adequate. The three things ITSM analytics needs at once — free text, a taxonomy, and an audit trail — are never published together, because the first two are confidential and the third is meaningless without them.

SOURCE	FREE TEXT	TAXONOMY	AUDIT TRAIL	VERDICT
UCI 498 — incident event log	anonymised codes	anonymised codes	**yes** — 141 712 events	lifecycle and time series; no NLP
`customer-support-tickets` (HF)	**yes**, multilingual	**yes**	no	NLP works; no flow, no series
Kaggle IT-support exports	yes	yes	no	same limitation

Decision. Support all of them behind one canonical schema, and add a **seeded synthetic generator** as the default so the project is runnable offline on day one — with ground truth for the parts that would otherwise be unevaluable.

> **THIS IS NOT A SHORTCUT AROUND REAL DATA**
> It is what makes the evaluation meaningful. The generator knows which tickets belong to which family (ground truth for clustering), which mood produced which wording (ground truth for sentiment), **where it planted each personal identifier** (ground truth for anonymisation, at character-offset precision) and **when it injected an anomaly** (ground truth for weak-signal detection, which is otherwise impossible to score at all).

WHAT THE GENERATOR SIMULATES

The reference dataset holds **7 897 tickets over 12 categories, in French and English**, driven by a catalogue of 12 families with slot-filled templates, assignment groups, per-family priority profiles and handling times.

- **Arrivals** — a Poisson process whose daily intensity is the product of a family weight (access & accounts is 17% of volume, databases 4%), a weekday profile (Monday +30%, Saturday −86%), a yearly profile (August trough, September peak), a growth trend (+12%/year), per-family effects (password resets ×1.9 on Mondays) and the injected anomalies.
- **Lifecycle** — a state machine per ticket: log-normal assignment delays and handling times scaled by priority; 8% of tickets misrouted and generating an extra reassignment; 25% passing through *Awaiting User Info*; 3% stuck 6–30× longer than normal (the vendor / spare-part / silent-user tail every desk carries); reopens drawn with a probability that grows with reassignments. Anything resolving after the extraction horizon stays **open**, reproducing the backlog a real extract contains.
- **Wording and mood** — the requester's mood is drawn from how badly the ticket went (reopened, breached, reassigned, critical) and *then* determines the phrasing. That direction matters: the model must recover the mood from the text alone, and the label is not a function of any feature it sees.
- **Difficulty** — the dataset deliberately injects **7% label noise, 13% ambiguous wordings and 12% typos**. Drawn from clean disjoint templates the data would be perfectly separable, every model would score near 1.0, and the benchmark would measure nothing.

THE LIMIT TO KEEP IN VIEW

A generator reproduces the structure its author encoded and nothing else. Real ticket text is messier, more multilingual, more sarcastic, and contains failure modes nobody anticipated. Every figure in section 3.3 is therefore a *relative* comparison between approaches on a common footing — not a forecast of production performance. The moment a real extract is available, `ITSM_DATASET=csv` points the same pipelines at it without change.

3.3 Component benchmarks

Each component of the architecture has its own experiment notebook: one question, several competing approaches measured on identical data, and one metric nominated in advance as the one that decides. The notebooks are the project's documentation of *why* each production choice was made.

A shared methodological rule runs through all five: **every approach runs in isolation**. Where a production configuration composes two techniques (the NER model with the rule engine, for example), the benchmark deliberately un-composes them, because a composed detector cannot be measured — a gap in one layer is covered by the other, and the score reports the union rather than the tool. Isolation is how the choice is made; composition is what ships.

#	COMPONENT	APPROACHES COMPARED	METRIC THAT DECIDES
1	**PII anonymisation**	rule engine · HF NER · Presidio · GLiNER (zero-shot)	residual personal values in the output
2	**Weak signals**	fixed threshold · robust baseline + EWMA · Prophet	recall *and* false alarms / 100 series-days
3	**Category classification**	majority · NB · logistic · SVM · embeddings+LR · LLM	macro-F1, then the coverage curve
4	**Sentiment**	lexicon · transformer · LLM	`negative` recall at usable precision
5	**Similar-case retrieval**	BM25 · dense · hybrid (RRF fusion)	hit rate@5

NOTEBOOK 1 — PII ANONYMISATION [MEASURED]

Question. Which detector removes the personal data, at what cost, and what does each one miss?

Protocol. Four detectors run standalone over the same sample. Detection is scored against character-offset ground truth; separately, the original values are searched for in the rewritten text — the only number that cannot be gamed by generous span matching.

MEASURED ON 2 000 TICKETS / 2 087 ANNOTATED VALUES	RULE ENGINE	HF NER
Coverage recall	**1.000**	0.999
Precision	**0.999**	0.976
F1	**0.9995**	0.987
Leak rate, direct identifiers	**0.000**	0.0012
Residual PII in the output	**0 / 2 087**	2 / 2 087
False-positive spans	**2**	50
Utility cost to the classifier (macro-F1)	**0.000**	−0.001

> **THE TABLE ABOVE IS RIGGED, AND SAYING SO IS NOT ENOUGH**
>
> The generator samples names from the same catalogue the rule engine's gazetteer is built from, so its `PERSON` recall is near-perfect *by construction*. A dedicated probe therefore re-runs each detector on twelve invented names spanning naming traditions no French/English list covers: **rule engine 0.667, transformer 0.944**. That 33% miss rate on real names is the number that decides deployment — not the headline F1.
>
> Presidio and GLiNER are implemented and wired into the same evaluation; their columns are *protocol*, not yet run.

Conclusion carried into the design. Not *pick one* but *formats from the rule layer, names from whichever model wins the unseen-name probe*. Structured identifiers (IBAN, email, national ID) are shapes and regex wins on them permanently; names are language and a model wins. The shipped transformer backend composes the two.

NOTEBOOK 2 — WEAK-SIGNAL DETECTION [MEASURED]

Question. Volume per family per day. Which detector raises the outage, how early, and how often does it cry wolf?

Protocol. Three detectors score the same series against the generator's anomaly manifest. Two metrics decide, and neither is accuracy: **false alarms per 100 series-days** (precision alone is misleading, because a detector firing for six days on one real outage looks precise) and **detection delay** (a detector that finds it on day 5 is a post-mortem tool).

MEASURED ON THE REFERENCE DATASET	RESULT
Injected anomalies caught	**4 / 4**
False alarms per 100 series-days	~1.7
Slow ramp — detection delay	12 days in

The slow ramp is the whole argument for the approach. A sudden spike is easy and any threshold catches it; a 15% daily drift where no single day is unusual can only be caught by an accumulating statistic, which is what the EWMA arm provides. The fixed-threshold and Prophet arms are implemented as challengers — *protocol, not yet run* — and the honest framing is that **if the fixed threshold matches the EWMA detector, the fixed threshold wins**: no dependency, and anyone can explain it at 3am.

The decision upstream of the detector is the series grain. Per category, per configuration item, per site? It sets the false-alarm rate before a line of code runs — too fine and every series is noise, too coarse and a real outage disappears into the total. The notebook sweeps it explicitly.

NOTEBOOK 3 — CATEGORY CLASSIFICATION [MEASURED]

Question. Does an LLM beat a linear model over n-grams, and by enough to pay for itself?

Protocol. Six approaches on an identical **temporal split** — train on the past, test on the most recent 20%. A random split flatters every model equally and predicts nothing about next month; the gap between the two *is* the drift estimate. Headline is macro-F1, never accuracy.

MEASURED ON 7 897 TICKETS / 12 CATEGORIES	RESULT
Calibrated LinearSVC over word+char TF-IDF — macro-F1	**0.864**
Majority-class baseline — macro-F1	0.027
Volume auto-routable above 0.8 confidence	~90%
Accuracy on that auto-routed volume	~93%

The majority baseline is in the table on purpose: a macro-F1 that does not clearly beat it means the pipeline is broken, not that the task is hard.

The coverage curve is the metric a service desk actually buys. Not *how accurate is the model* but *what share of tickets can we auto-route at 95% accuracy, and how many still need a human*. This is why calibration matters more than the last point of macro-F1, and why an approach that cannot produce a comparable confidence is not comparable on the decisive axis.

> **ON THE LLM ARM**
> It is implemented with the answer constrained to the taxonomy by a JSON schema, so the error rate measures the model rather than a parser, and it is asked for a confidence so it can appear on the coverage curve. It is *protocol*: gated behind an explicit flag because it is the only component that leaves the machine and the only one with a per-ticket bill, which the notebook prices before spending anything. Where an LLM genuinely wins and this table will not show it is the **cold start** — with no labelled history the linear arms cannot be trained at all.

NOTEBOOK 4 — SENTIMENT [MEASURED]

Question. Which analyser finds the frustrated requester early enough to act?

Protocol. This is a *triage signal*, not an analytics number, and the cost is asymmetric: a missed frustrated requester is an escalation, a false positive is thirty seconds of an agent's attention. The headline is therefore **recall on** `negative` at a precision the desk can live with. Kappa is the aggregate, because the distribution is heavily neutral and accuracy would reward an analyser for saying neutral to everything.

MEASURED — LEXICON ANALYSER	RESULT
Macro-F1	0.804
Cohen's kappa	0.664
Recall on frustrated requesters	**87%**
Precision at that recall	61%

> **THE CEILING HERE IS THE LABEL, NOT THE MODEL**
>
> There are **no human sentiment labels**. The reference is a weak label derived from signals correlated with frustration — reopen count, escalation, breached SLA, reassignment churn — not from anyone reading the ticket. Every score measures *agreement with that heuristic*, and an analyser can only look good by resembling it. Before this drives anything user-visible, 300 tickets should be annotated by hand, by two annotators, with an inter-annotator agreement figure: that is the real ceiling, and without it there is no way to know whether a kappa of 0.66 is good.

NOTEBOOK 5 — SIMILAR-CASE RETRIEVAL [PROTOCOL]

Question. Put the K resolved tickets closest to a new one in front of the agent. Which retriever finds them?

Protocol. Corpus is resolved tickets from the past, queries are later tickets — a random split would let a retriever find a near-duplicate filed the same afternoon. Fusion is on *ranks*, not scores: BM25 is unbounded and cosine lives in [−1, 1], so normalising them onto a common scale means inventing a mapping that changes with every query. Reciprocal-rank fusion uses only position.

The headline is **hit rate@5** rather than recall@5: the agent sees five cases, and whether that panel contained anything useful is the experience; recall over a relevant set of 200 same-category tickets is not.

> **TWO CAVEATS THAT OUTRANK THE NUMBERS**
>
> **The relevance labels are a proxy.** Nobody has judged which resolved ticket is genuinely useful for a new one, so a case counts as relevant when it shares the new ticket's category and subcategory. That rewards topical similarity, which is not the same as a resolution that transfers.
>
> **The load-bearing question is upstream.** Retrieval and "suggest a mitigation" both assume a populated resolution field. In most real ITSM exports close notes are empty or boilerplate. **Sampling 200 closed tickets and reading the close notes is a one-day check that decides roughly a quarter of the work** — and it has to happen before any of this is built.

3.4 Limits

They fall into four groups, ordered by how much they could change the plan.

A. DATA — THE BINDING CONSTRAINT

- **No client data has been seen.** Everything measured here runs on a synthetic corpus. The figures are valid as relative comparisons between approaches; they are not a forecast of production performance.
- **Resolution text may not exist.** If close notes are empty or boilerplate — the common case — the retrieval and suggestion components have nothing to work with regardless of retriever quality.
- **The ticketing tool, exportable fields and history depth are unknown**, along with whether any categorisation already exists that could serve as a weak label. These are open items on the project's Questions page.
- **Confirmed constraint:** tickets do contain personal data (first names at minimum), which is what makes the anonymisation gate mandatory rather than optional.

B. METHOD

- **No human labels for sentiment**, and **no human relevance judgements for retrieval.** Both are scored against proxies, and both proxies could be wrong in the same direction as the models that fit them.
- **The unseen-name probe is small and synthetic.** Twelve names in six sentence frames proves a direction, not a number.
- **One prompt, one embedding model, one fusion constant** were tested where several should be swept. Prompt wording is usually a larger lever than model choice, and no sweep has been run.
- **Prophet is scored in-sample** in the weak-signal comparison, which flatters it relative to the causal detectors it competes with.

C. ARCHITECTURE AND OPERATIONS

- **The multilingual constraint is real and under-priced.** FR/EN mixed text forces a multilingual embedding model, a bilingual PII detector and a language-aware lexical index. Presidio in particular is configured for one language at a time, and this corpus is two.
- **No deployment view yet:** volumes, per-hop latency budget, data contracts with the ITSM tool, and who is on call for a false alert are all undefined. An alert nobody routes is not an alert.
- **Weak-signal preconditions are hard:** 6–8 weeks of history per series before any threshold means anything, and enough volume per series that a Poisson rule is not firing on two tickets.
- **The prediction horizon and acceptable false-positive rate have not been agreed with the client** — and these two numbers determine the design of the detector, not the reverse.

D. SCOPE

- **Three products are currently entangled.** Agent assist, manager analytics and API integration have different buyers and different success criteria. Either of the two pipelines alone is a deliverable; both at once is a programme.
- **Phase 2 depends on Phase 1 in a hard sense.** Weak-signal detection on families that have not been stabilised and validated with the team measures noise, however good the detector.

4. Conclusion

The work delivers a complete, runnable design for structuring the ticket flow, with each component's technical choice justified by a measurement rather than by convention — and with the boundary between what is proven and what is assumed drawn explicitly.

What is established

The approach is viable and the cheap methods go further than expected. Category classification reaches macro-F1 0.864 against a 0.027 baseline, with around 90% of volume auto-routable at ~93% accuracy — using a calibrated linear model over TF-IDF that trains in seconds on CPU and can be asked which n-grams drove a decision. Weak-signal detection catches all four injected anomalies at roughly 1.7 false alarms per 100 series-days and flags a slow ramp twelve days in. Anonymisation leaves zero of 2 087 personal values in the output at no measurable utility cost to the downstream classifier.

The evaluation protocol is the real asset. Ground truth for anomalies, character-offset ground truth for PII, an honest temporal split, a coverage curve that converts a model score into an operational decision, and an unseen-name probe that deliberately breaks the benchmark's own favourable bias. These transfer to the client's data unchanged; the models will be retrained, the protocol will not.

What must happen next, in order

1. **Answer the data questions.** Tool, export format, available fields, history depth, existing categorisation. Nothing downstream can be sized without them.
2. **Sample 200 closed tickets and read the close notes.** One day of work that determines whether the retrieval and suggestion components are buildable at all.
3. **Agree the two numbers that design the detector:** the prediction horizon for a load alert, and the acceptable false-positive rate.
4. **Annotate a few hundred real tickets** — PII spans and sentiment, with two annotators — to replace the synthetic ground truth where it matters most.
5. **Run the five benchmarks on the real extract** and pick per component. The comparison harness is written; only the data is missing.
6. **Ship one product first.** The recommendation is agent-side classification with abstention: it is the most mature component, it has the clearest metric, and it generates the labelled feedback every other component depends on.

> **THE HONEST SUMMARY**
> The architecture is designed, the components are implemented, the evaluation protocol is sound and partially executed. What is not yet established is anything about *this client's* tickets — and the sequence above is deliberately ordered so that the cheapest checks come first and can still change the plan.

Sources: project Notion space (Problem statement, Solutions & Results, Questions, and the two literature review documents) and the `itsm-insights` repository (`docs/01-data.md`, `03-modeling.md`, `06-results.md`, `07-privacy.md`, and the five experiment notebooks under `notebooks/`).